ZINGSA Collector API & Dynamic Form System Documentation (Draft)

NB: This documentation is intended for developers, backend engineers, GIS specialists, and system integrators involved in implementing or extending the ZINGSA Collector platform.

1. Introduction

ZINGSA Collector is a mobile GIS data collection application built with React Native and Expo. It supports both form-first workflows (traditional sequential form workflows similar to KoboCollect) and map-first workflows designed for GIS data collection involving points, lines, and polygons. The platform supports a wide range of operational workflows, including:

Dynamic Form Features

- Dynamic question rendering
- Conditional logic workflows
- Validation rules and constraints
- Configurable form structures
- Offline-compatible submissions

GIS & Spatial Features

- Map-first GIS workflows
- Geometry collection and editing
- Spatial attribute capture
- Point, line, and polygon workflows
- GIS-enabled mobile data collection

Media & Capture Features

- Image and photo capture
- Video recording
- Voice/audio recording
- File uploads
- Signature capture

This documentation defines the technical specification for:

- API communication structures
- Dynamic form schemas
- GIS workflow behavior
- Validation configuration
- Conditional logic structures
- Synchronization workflows
- Dynamic rendering behavior

- Expected submission payload structures

2. System Architecture

Below is a simple overview of the components of the ZINGSA Collect system.

Component	*Responsibility*
Mobile Application	Dynamically renders forms, supports offline collection, GIS workflows, media capture, and synchronization
Frontend Form Builder	Enables visual creation, editing, management, and configuration of forms
Backend API Services	Serves form definitions, stores submissions, manages synchronization, media, and users
Database Layer	Stores forms, versions, submissions, media references, and synchronization metadata

2.1 Frontend Form Builder Responsibilities

The ZINGSA Collect frontend form builder enables users to create, configure, and manage dynamic, JSON-driven data collection workflows. It will also have an admin panel for user/organizations management.

The frontend form builder will support:

- Creating, editing, and deleting forms
- Adding and managing dynamic question structures
- Reordering and organizing form questions
- Configuring validation rules and field constraints
- Configuring conditional logic workflows
- Configuring GIS workflow modes
- Configuring geometry capture types
- Configuring media capture settings
- Generating valid JSON form definitions for mobile rendering
- Managing form versions and workflow configurations
- User management

2.2 Backend Responsibilities

The backend system will be responsible for the following:

- Receiving and managing forms created through the frontend form builder
- Serving dynamic JSON form definitions to mobile clients
- Managing user accounts and submitted field data
- Storing form submissions
- Storing media uploads and attachments
- Managing form versions and revisions
- Supporting offline synchronization workflows
- Providing validation and conditional logic configurations

Admin Responsibilities

- Organizational management
- User management
- Role management
- Form permissions
- Submission monitoring
- Synchronization monitoring
- Audit logs

2.3 Mobile Application Responsibilities

The ZINGSA Collector mobile application is responsible for dynamically rendering forms, managing field data collection workflows, and supporting offline GIS-enabled operations.

The mobile application:

- Detects and processes form workflow modes
- Dynamically renders all supported question types
- Applies validation rules and field constraints
- Processes conditional logic and visibility rules
- Handles GIS and spatial data collection workflows
- Captures and synchronizes media attachments
- Validates required fields before submission
- Manages offline data storage and synchronization
- Supports offline-first mobile data collection workflows

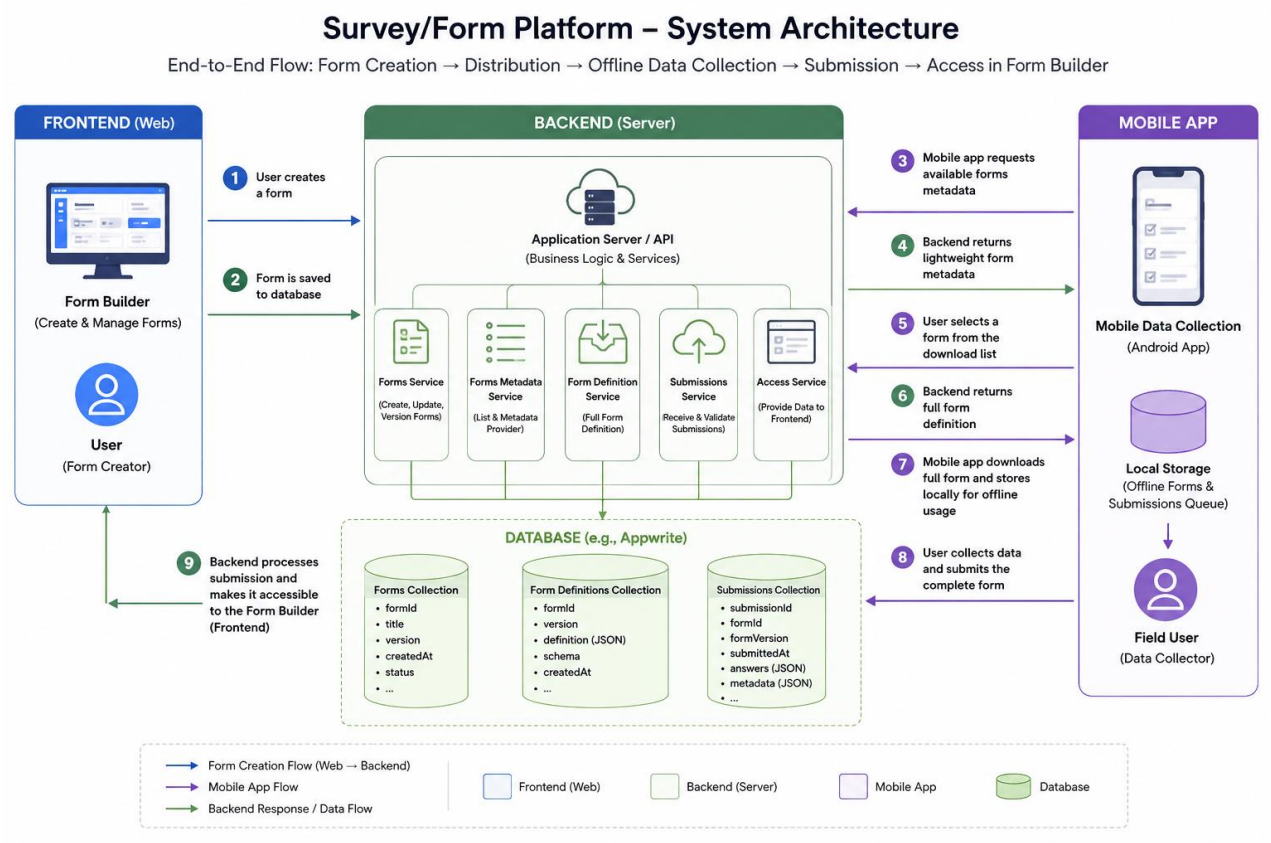
3. API Overview

All API communication uses JSON. For media uploads, the proposed flow is that the mobile application first submits media files to a bucket, then attaches the file bucket Id/url to the form and submits the form to the backend server.

```
Content-Type: application/json
Accept: application/json
```

```
{
  "success": true,
  "message": "Request successful",
  "data": {}
}
```

3.2.1 General Workflow



The general workflow is:

1. User creates a form in the Form Builder (Frontend)
2. Form is sent and saved to the database (Backend)
3. Mobile app requests available forms metadata
4. Backend returns lightweight form metadata
5. User selects a form from the download list.
6. Backend returns full form definition.
7. The mobile app downloads the full form and stores it locally for offline usage.
8. User collects field data and submits the completed submission.
9. Backend processes the received submission and makes it available within the Form Builder frontend.

3.2.2 Available Forms Endpoint

The mobile app uses the following endpoint to populate the Downloads screen. This endpoint returns only a lightweight version of the available forms.

Endpoint

GET /api/forms/available

Example Request from the mobile app

GET /api/forms/available?page=${page}&limit=${limit}&search=${encodeURIComponent(searchQuery)}
GET /api/forms/available?page=${page}&limit=${limit}

Query Parameters

Parameter	Type	Description
page	number	Pagination page number
limit	number	Number of forms per page
projectId	string	Filter forms by project
search	string	Search forms by title/category

Available Forms Response Example

The backend system must return the following response when the mobile application requests for available forms.


```
{
  "forms": [
    {
      "id": "test_form_2",
      "title": "Wildlife Observation",
      "description": "Form for recording wildlife sightings and observations",
      "version": "v2.2",
      "mode": "form_first",
      "category": "Wildlife",
      "lastUpdated": "2024-01-20",
      "checksum": "sha256:xyz789uvw456",
      "projectId": "wildlife_project_002",
      "createdAt": "2026-01-20T12:34:56Z"
      "canDownload": true,
      "fileEtag": "W/\"test_form_2-v2\"",
      "formRevision": 2
    }
  ],
  "pagination": {
    "limit": 10,
    "page": 1,
    "offset": 0,
    "total": 30,
    "hasMore": true,
    "currentPage": 1,
    "totalPages": 3
  }
}
```


Note: The ‘canDownload’ field may later be replaced by permission-based authorization workflows.

Explanation of Metadata Field Definitions Descriptions

Field	Description
id	Unique form identifier used for downloading full forms
title	Human-readable form title
description	Form description shown to users
version	Public form version
mode	Either form_first or map_first
category	Organizational grouping/category
checksum	Used for offline integrity verification
projectId	Associated project identifier
createdAt	Form creation timestamp
canDownload	Indicates whether download is allowed
fileEtag	Used for HTTP caching and synchronization
formRevision	Internal revision tracking number

3.3 Downloading Full Forms

After the user selects a form from the available downloads screen, the mobile application downloads the full form definition using the form ID.

Endpoint

GET /api/forms/{formId}

Example Request

GET /api/forms/test_form_2

3.4 Supported Form Modes

The mode field can contain either:

- form_first
- map_first

The mobile application uses this value to determine:

- Rendering workflow
- GIS behavior
- Offline synchronization logic
- Map interaction behavior

3.5 Submit Form Data

Submission payload structures will be documented later.

4. Form Modes

Mode	*Description*
form_first	Traditional sequential form workflow like kobocollect/ epicollect
map_first	GIS-focused multi-feature collection workflow

4.1 Form-First Workflow

The form_first mode is the standard form workflow used for general-purpose data collection.

In this workflow:

• Questions are displayed sequentially

• GIS questions are embedded inside the form

• Each GIS question captures a single feature independently

4.2 Map-First Workflow

The map_first mode is primarily designed for GIS-focused workflows involving multiple spatial features.

Typical workflow:

1. User opens the map
2. User captures a GIS feature
3. Attribute form appears
4. User saves feature
5. User continues capturing additional features without leaving the map and selecting the form again.

5. Complete Form-First Example


```
{
 "formId": "test_form_2",
 "title": "Wildlife Observation",
 "description": "Comprehensive wildlife sighting form with conditional questions",
 "version": "v2.1",
 "mode": "form_first",
 "projectId": "PROJ-WILDLIFE-002",
 "createdBy": {
  "name": "Wildlife Conservation Team",
  "organization": "Environmental Protection Agency"
 },
 "createdDate": "2026-01-20T10:15:30Z",
 "questions": [
  {
   "id": "species_name",
   "type": "text",
   "label": "Species Name",
   "required": true,
   "placeholder": "Common or scientific name",
   "minLength": 2,
   "maxLength": 100
  },
  {
   "id": "observation_date",
   "type": "date",
   "label": "Observation Date",
   "required": true
  },
  {
   "id": "behavior",
   "type": "radio",
   "label": "Behavior Observed",
   "required": true,
   "options": [
    {"value": "feeding", "label": "Feeding"},
    {"value": "resting", "label": "Resting"},
    {"value": "other", "label": "Other"}
   ]
  }
 ]
}
```


NB: If the mode field is omitted, the mobile application defaults to form_first behavior.

6. Complete Map-First Example


```
{
 "formId": "test_form_6",
 "title": "GIS Polygon Collection",
 "description": "Map-first form for capturing GIS polygons with attributes.",
 "version": "v1.0",
 "mode": "map_first",
```

```
  "geometryType": "polygon",
  "projectId": "PROJ-GIS-POLYGONS-006",
  "questions": [
    {
      "id": "location_name",
      "type": "text",
      "label": "Location Name",
      "required": true
    },
    {
      "id": "habitat_type",
      "type": "dropdown",
      "label": "Habitat Type"
    }
  ]
}
```

NB: If the mode field is omitted, the mobile application defaults to form_first behavior.

7. Form Structure Specification

Field	*Description*
formId	Unique identifier for the form
title	Human-readable form title
description	Additional information about the form
version	Used for updates and syncing
mode	Controls application behavior
geometryType	Controls GIS capture workflow
questions	Contains all question definitions

8. Question Structure Specification

```
{
  "id": "species_name",
  "type": "text",
  "label": "Species Name",
  "required": true
}
```

9. Supported Question Types

The following are the question types that are currently being supported by the mobile app.

Type	*Description*
text	Single-line text input
textarea	Multi-line text input
email	Email input
phone	Phone number input
url	URL input
number	Numeric input

date	Date picker
time	Time picker
radio	Single-choice selection
checkbox	Multiple-choice selection
dropdown	Dropdown selection
image	Image/photo capture
video	Video capture
voice	Voice recording
signature	Signature capture
location	GPS location capture
polygon	GIS polygon drawing
point	GPS point capture
line	GIS line capture
polygon	GIS polygon capture
audio	Audio recording
file	Generic file upload
barcode	Barcode scanner
qr	QR scanner

10. Validation Rules

A user can add validation/restrictions to each question using the max and min values.

```
{
  "minLength": 2,
  "maxLength": 100
}
```

```
{
  "min": 0,
  "max": 100,
  "numericType": "decimal",
  "decimalPlaces": 2
}
```

11. Conditional Logic

To implement the conditional logic of what happens when conditions of a question are met or not, the conditional operators are used. These are helpful for cases when the user wants question 2 to be skipped if the user has clicked no in the previous question, etc.

```
{
  "condition": {
    "field": "behavior",
    "operator": "equals",
    "value": "other"
  }
}
```

Operator	***Description***
equals	Exact value match
not_equals	Opposite exact match

contains	Checks if the array/string contains the value
not_contains	Opposite contains a match

12. Media Questions

For questions where the user is supposed to upload some media files, the form must have a type of media file and also the max or min number of required files.

```
{
  "type": "image",
  "maxPhotos": 3
}
```

13. GIS Workflows

Geometry	***Description***
Point	Single GPS point
line	Multi-point GIS line
polygon	Closed GIS polygon

14. Dynamic Rendering Behavior

The mobile application dynamically renders forms entirely from backend JSON responses. The application automatically detects form modes, renders questions, applies validation rules, handles GIS workflows, and processes media uploads.

15. Frontend Form Builder Requirements

The frontend form builder must support:

- Creating forms
- Editing forms
- Reordering questions
- Configuring conditions
- Configuring GIS workflows
- Configuring media settings

16. Expected Submission Output Structures

This will be added later

17. Best Practices

Labels

- Keep labels short
- Include units where needed
- Avoid technical jargon

Validation

- Use minimum and maximum constraints
- Avoid unnecessary required fields
- Use clear validation messages